

Codex Token Meter

本地长期额度与速度监测工具

3099404236^{1*}

¹Independent developer

Codex 的界面通常只显示额度百分比，难以直接回答“最近一个百分点经过了多少 Token”以及“当前工作负载的真实速度是多少”。本文介绍 **Codex Token Meter**：一个只在本机运行的长期监测工具。它融合 **Codex App Server** 的额度与账户汇总、本地会话 **Token** 事件和任务完成耗时，持续构造额度边界样本，并把首段等待、整轮有效速度与开始响应后的速度分开呈现。工具不会为了采样主动请求模型；后台仅绑定本机回环地址，真实账户遥测不进入公开仓库。本文给出指标定义、实现架构、复现方式、安全边界与局限。

Codex | Token monitoring | local telemetry | performance measurement

Project repository: github.com/3099404236/codex-token-meter

问题与目标

Codex 的额度窗口用“已使用百分比”描述剩余能力，而模型和任务本身以 Token、等待时间和完成时间表现成本与速度。这两套口径之间没有公开的固定换算。一个百分点可能覆盖不同数量的输入、输出、缓存与推理 Token；长上下文、工具调用、模型选择和服务调度也会改变用户体验。

本工具不试图推断一个永久不变的兑换率，而是解决三个更可操作的问题：

- 当前额度窗口已使用和剩余多少，何时重置？
- 最近一次或一段时间内，每个观测到的百分点经过了多少总 Token？
- 正常任务从启动到完成的有效速度如何，慢在首段等待还是中途执行？

设计目标是“被动、长期、本地、可解释”。采集器不主动制造额外模型负载，浏览器关闭后继续运行；所有记录留在用户机器上，并明确区分官方返回值、实时本地计数与经验估算。

系统架构

实现由一个 Node.js 入口程序和一个无第三方前端依赖的本地面板组成。入口程序启动 Codex App Server 子进程，通

过标准输入输出交换 JSON-RPC 消息，同时增量读取 Codex 本地会话 JSONL。HTTP 服务只监听 127.0.0.1:8765。

服务启动时会回填最近 7 天内可读取的已完成任务，回填总量设有上限，避免一次载入超大历史。之后每 2 秒检查活跃会话增量，每 30 秒刷新活跃文件集合，每 60 秒采样额度。任务 ID 用于跨重启去重；额度、回合和服务事件以 JSONL 追加保存。

指标定义

额度与每 1% Token.

设第 t 次采样的官方已用比例为 u_t ，本地累计总 Token 计数为 C_t 。剩余比例为

$$r_t = 100 - u_t.$$

对同一额度窗口中两个连续观测边界 a 与 b ，经验 Token/百分点定义为

$$q_{a,b} = \frac{C_b - C_a}{u_b - u_a}, \quad u_b > u_a.$$

当 $u_b - u_a = 1$ 时，记录被标记为“精确跨 1%”；若跳过多个百分点，则只报告该区间的平均值。这里的“精确”指采样边界恰好相邻，不代表服务端存在固定的 Token 配额公式。

总 Token 的本地口径是输入 Token 加输出 Token。缓存输入属于输入 Token 的子集，不重复累加；输出侧包含可见输出和推理输出。账户 lifetime 汇总可能延迟，因此工具优先用本地计数形成实时样本。若计数器没有变化，不会生成“1% 等于 0 Token”的伪样本。

速度与首段等待.

对于一个完成回合，设可见输出 Token 为 V ，整轮耗时为 T ，首段等待为 F 。整轮有效速度与开始响应后的速度分别为

$$s_{\text{effective}} = \frac{V}{T}, \quad s_{\text{active}} = \frac{V}{T - F}.$$

层次	数据源或组件	职责
官方状态	account/rateLimits/read	额度窗口、已用百分比、重置时间
账户汇总	account/usage/read	lifetime Token；允许延迟或暂时不可用
本地会话	~/codex/sessions	Token 计数、首段等待、整轮耗时、任务完成
采集服务	codex-meter.mjs	去重、回填、聚合、持久化与安全关闭
展示层	dashboard/	时间范围、额度曲线、1% 柱状图和速度折线

表 1 数据来源与组件分工。账户汇总用于长期参考，本地会话计数用于更及时的百分点样本。

指标	回答的问题	主要局限
首段等待	“为什么迟迟不开始回复?”	包含调度和前置推理
整轮有效速度	“这个任务总体推进得多快?”	工具型任务会显著拉低
开始响应后速度	“开始以后整体推进得多快?”	仍包含工具和停顿
纯文字流速	“连续吐字阶段有多快?”	不代表真实复杂任务

表 2 四种速度口径不能互相替代。默认面板优先呈现真实工作负载。

整轮有效速度把推理、工具运行、用户可见停顿和网络等待全部计入,更接近真实工作体感。开始响应后的速度扣除了首段等待,但仍不等于纯文字流式阶段的瞬时生成速度。面板另外保留主动纯文字测速命令,避免把两种场景混为一谈。

本地面板与运行方式

在已安装并登录 Codex CLI 的 Windows 环境中,进入 code/ 后运行:

```
node .\codex-meter.mjs open
```

该命令启动分离的后台进程并打开本地网页。再次执行会检查健康端点并复用现有进程,不会重复采集。关闭网页不影响后台;显式停止命令为:

```
node .\codex-meter.mjs stop
```

面板提供 30 分钟、2 小时、6 小时、24 小时、7 天和全部历史范围。额度曲线显示已使用比例;柱状图显示每个跨档样本;速度图同时显示整轮有效速度和开始响应后的速度。表格保留原始完成时间、可见输出、首段等待和样本口径,便于追溯聚合数值。

为了便于 Windows 用户使用,项目附带“打开长期监测面板”和“停止长期监测”两个双击脚本。系统重启后需要重新启动采集器,工具当前不会擅自添加开机启动项。

验证与可复现性

发布前采用以下检查,而不公开作者账户的实际数值:

- 1. 用 `node --check` 检查采集器与前端脚本语法;
- 2. 启动服务并验证静态资源、健康接口和不同时间范围 API 均返回成功;
- 3. 重复执行 `start`, 确认 PID 不变;执行 `stop` 后立即重启, 确认得到新 PID;
- 4. 从历史会话回填速度点,并验证同一 `turn ID` 不会重复写入;
- 5. 模拟或观察 `lifetime` 汇总暂不更新的情形, 确认零增量被忽略,本地计数成为优先来源;
- 6. 重新编译本报告,并逐页渲染检查中文字体、表格、公式和页码。

仓库不附带真实遥测数据。复现者只需在自己的 Codex 环境运行程序,数据会写入代码目录中的 JSONL 和本地状态文件。公开代码的验证重点是数据流、口径与故障处理,而不是复制某个账号的具体额度结果。

安全与隐私边界

本地日志可能包含用量、任务时间、会话标识和性能特征,应被视为私密数据。项目的 `.gitignore` 排除基线、服务状态、速度历史、监听日志和私用密钥文件。发布过程还会检查敏感文件是否被 Git 跟踪。

HTTP 面板只绑定回环地址;停止接口需要本地状态文件中的随机令牌。工具不上传遥测,也不向局域网开放端口。然而,同一操作系统用户下的其他进程通常仍可读取用户文件,因此该设计不是针对本机恶意软件的隔离方案。

局限与后续工作

第一,额度百分比是离散且可能延迟的观测,60 秒采样仍可能跨过多个百分点。第二,Token 总量与实际额度扣减未必线性;服务端可能综合模型、缓存和其他权重。第三,本地会话事件格式属于实现接口,未来 Codex 版本可能调整字段。第四,工具只能观察客户端可见数据,不能证明后台实际路由到哪个模型。第五,复杂工具任务的低有效速度不等于文本模型本身生成慢。

后续可以增加可配置采样频率、开机启动、数据导出、异常点说明和多额度窗口并列显示。任何自动上传或远程访问能力都应保持默认关闭,并由用户显式启用。

结论

Codex Token Meter 把“额度还剩多少”“最近 1% 大约经过多少 Token”和“真实任务为什么慢”放进同一个本地长期视图。它不宣称破解服务端配额算法,而是以透明的数据来源、明确的统计口径和可追溯样本改善用户对消耗与速度的判断。对这类不完全可观测的系统,持续测量和保留不确定性比给出一个看似精确的固定兑换率更可靠。

项目主页: github.com/3099404236/codex-token-meter
Codex App Server 参考: learn.chatgpt.com/docs/app-server